UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

CORSO DI LAUREA TRIENNALE IN
INFORMATICA

Insegnamento:
Basi di dati I

Professore

Silvio Barra

Traccia 1: Gestore ToDo

Nunzio Grasso

Matricola: N86005509

Anno Accademico 2025/2026

INDICE

Anno Accademico 2025/2026 1

1. Introduzione 4

1.1 Traccia 4

1.2 Analisi del problema 5

2.Progettazione Concettuale 6

2.1 Class Diagram 6

2.3 Ristrutturazione del Class Diagram 7

2.3.1 Analisi delle ridondanze 7

2.3.2 Eliminazione di attributi composti e multivalore 7

2.3.3 Trattamento delle gerarchie 7

2.4 Dizionario delle classi 7

2.5 Dizionario delle associazioni 9

2.6 Dizionario delle enumerazioni 11

2.7 Dizionario dei vincoli 11

3.Progettazione Logica 14

3.1 Mapping delle associazioni 14

3.2 Schema logico 15

4.Progettazione Fisica 15

4.1 Definizione Tabelle 15

4.1.1 Definizione della Tabella users 15

4.1.2 Definizione della Tabella boards 16

4.1.3 Definizione della Tabella todos 16

4.1.4 Definizione della Tabella shared_todos 17

4.2 Definizione Viste 18

4.2.1 Definizione della vista v_user_visible_todos 18

4.3 Definizione Trigger 19

4.3.1 Inizializzazione automatica delle bacheche (Vincolo Inizializzazione Account) 20

4.3.2 Gestione automatica del posizionamento (Vincolo Integrità Posizionamento) 20

4.3.3 Gestione dell'ordine nelle condivisioni (Vincolo Integrità Posizionamento) 22

4.3.4 Allineamento automatico dei contenitori condivisi (Vincolo Allineamento Automatico Bacheche) 23

4.3.5 Sincronizzazione dinamica allo spostamento (Vincolo Allineamento Automatico Bacheche) 24
4.3.6 Gestione delle eliminazioni delle condivisioni (Vincolo Eliminazione a Cascata dei todo condivisi) 26
4.3.7 Inibizione della modifica del titolo (Vincolo Inibizione Renaming Bacheche Condivise) 27
4.3.8 Propagazione della ridenominazione (Vincolo Allineamento Automatico Bacheche) 28
4.3.9 Aggiornamento automatico dei timestamp di audit 29
4.4 Definizione Funzioni 30
4.4.1 Ricerca testuale (fn_search_todos) 31
4.4.2 Filtro scadenze giornata odierna (fn_get_due_today) 31
4.4.3 Ricerca per data limite (fn_get_due_within) 31
4.5 Note sulla gestione dell'ordinamento dei todo in una bacheca 32

1. Introduzione

Il presente documento ha lo scopo di mostrare le fasi di analisi, modellazione e realizzazione di un sistema informativo relazionale basato su DBMS PostgreSQL progettata per una piattaforma dedicata alla gestione di attività. L'elaborato è stato prodotto dallo studente Grasso Nunzio a fini valutativi per il corso di Basi di Dati I presso l'Università degli Studi di Napoli "Federico II".

1.1 Traccia

Si sviluppi un sistema informativo, composto da una base di dati relazionale e da un applicativo Java dotato di GUI (Swing) che consente di tenere traccia delle attività personali da svolgere (nel seguito chiamate semplicemente "*ToDo*"), ispirato al software Trello.
Il software deve consentire all'utente di organizzare e gestire le proprie attività personali in modo efficiente, utilizzando un'interfaccia intuitiva e flessibile simile a quella di Trello (https://trello.com/).
L'utente entra nel sistema specificando la propria login e password, che devono essere univoche.
I ToDo sono organizzati in tre bacheche (ogni bacheca ha un titolo e una descrizione), che possono essere create, modificate ed eliminate. I titoli delle tre bacheche sono Università, Lavoro e Tempo Libero.
I ToDo all'interno di una bacheca sono ordinati, secondo un ordine modificabile dall'utente.
L'utente può creare, modificare ed eliminare ToDo, così come può spostare un ToDo da una bacheca all'altra oppure cambiarne la posizione all'interno della bacheca.
Ogni ToDo ha un titolo e una data di scadenza, un link ad una URL correlata all'attività, una descrizione dettagliata e un'immagine. Tutti questi elementi sono opzionali e possono essere modificati in qualsiasi momento.
Ogni ToDo può contenere inoltre una lista di altri utenti che condividono quel ToDo. In pratica quel ToDo comparirà nella bacheca corrispondente di ognuno di tali utenti . Ad esempio, se Pippo, Pluto e Paperino condividono un ToDo, ed esso si trova nella bacheca

Tempo Libero di Pippo, allora esso comparirà anche nelle bacheche Tempo Libero di Pluto e Paperino. Ogni utente può leggere chi sono gli altri utenti con i quali il ToDo è condiviso. L'autore del ToDo può aggiungere o eliminare condivisioni.
Infine il ToDo ha un colore di sfondo che viene mostrato nell'interfaccia grafica. L'utente deve poter scrivere e modificare ognuna di tali informazioni. Un ToDo può essere settato come completato oppure come non completato (di default è non completato).
Il sistema deve poter fornire su richiesta dell'utente, l'elenco di ToDo in scadenza nella giornata odierna, oppure quelli in scadenza entro un certo giorno specificato dall'utente. Il sistema deve consentire anche la ricerca per nome o per titolo dei ToDo.
Per i ToDo scaduti, il sistema mostra il nome del ToDo in rosso, per evidenziare il superamento della scadenza prevista.

1.2 Analisi del problema

Il sistema informativo progettato mira alla gestione strutturata e collaborativa delle attività personali (ToDo). L'obiettivo primario è garantire la persistenza di dati organizzati all'interno di contenitori logici denominati "Bacheche". Dallo studio della traccia e dalle specifiche richieste, l'analisi si articola nei seguenti punti:

- **Gestione Utenti e Accessi:** La base di dati deve gestire profili utente identificati da credenziali univoche (username e password). Ogni utente deve poter operare in un ambiente isolato o condiviso, garantendo la riservatezza dei propri dati.

- **Struttura delle Bacheche:** Il sistema prevede l'esistenza di bacheche con titoli predefiniti (*Università, Lavoro, Tempo Libero*), ma deve consentire all'utente la flessibilità di creare, modificare o eliminare ulteriori contenitori. Ogni bacheca funge da raggruppamento logico per i ToDo.

- **Caratteristiche dei ToDo:** Ogni attività deve poter memorizzare un set eterogeneo di informazioni: titolo, descrizione dettagliata, data di scadenza, link URL e un riferimento a un'immagine. La progettazione deve prevedere l'opzionalità di questi elementi, permettendo al contempo la loro modifica in qualsiasi momento.

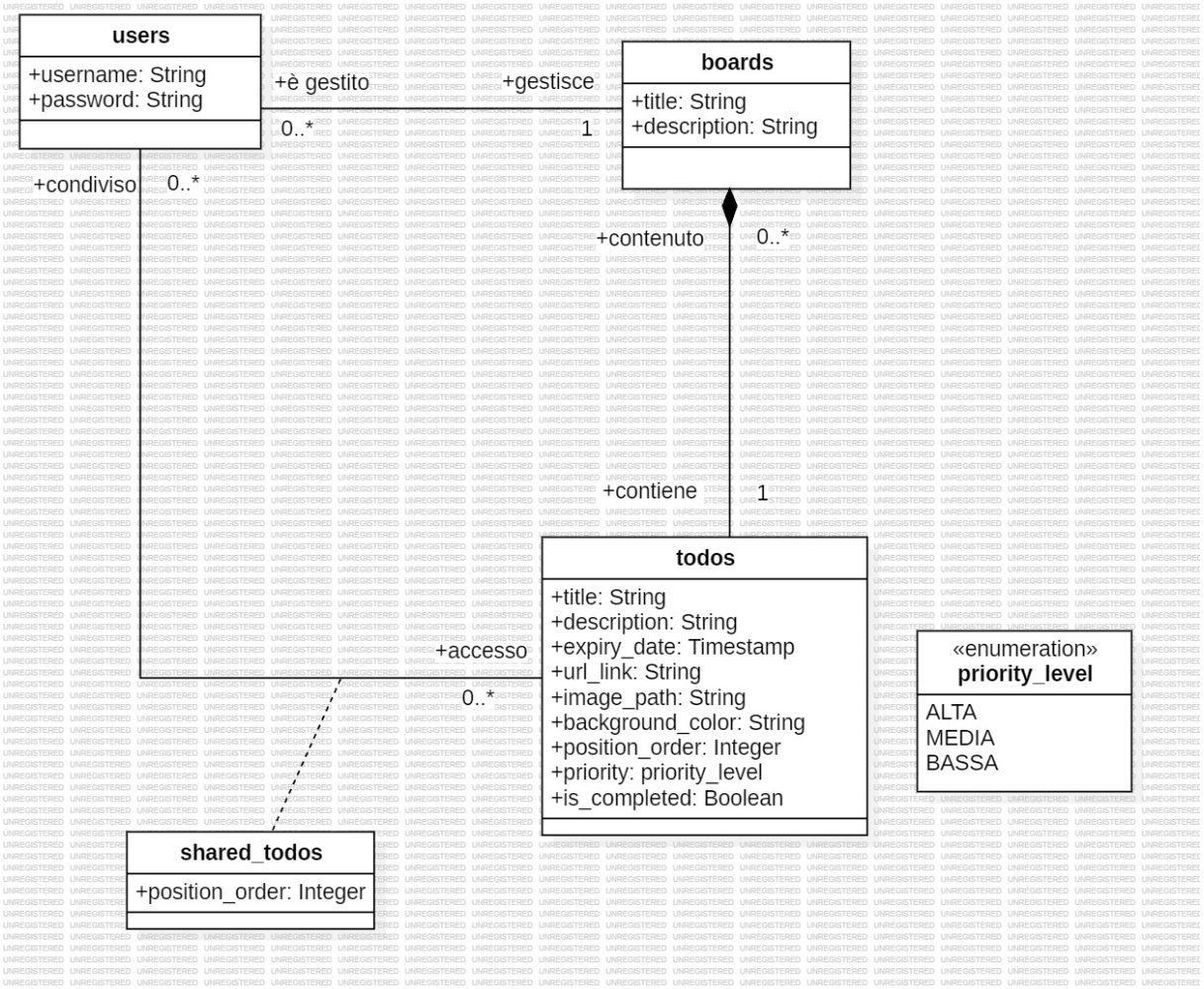
- **Logica di Condivisione e Permessi:** Il sistema deve supportare la collaborazione multi-utente. Un ToDo può essere condiviso con una lista di altri utenti; in tal caso, l'attività apparirà automaticamente nella bacheca omologa di ogni collaboratore. La condivisione dipende strettamente dal titolo della bacheca del proprietario. Si assume quindi che il titolo della bacheca dell'utente a cui è stato condiviso non possa essere modificato per non perdere la condivisione. Si assume inoltre che un todo condiviso possa essere eliminato solo dal proprietario, mentre un collaboratore può soltanto eliminare la sua condivisione, cosa che può fare anche il proprietario.

• **Dinamicità e Monitoraggio:** La base di dati deve supportare un sistema di ordinamento personalizzato per i todo all'interno della bacheca. Inoltre, deve fornire meccanismi per il filtraggio temporale (scadenze odierne o entro date specifiche) e per la ricerca testuale.

2.Progettazione Concettuale

In questa sezione viene definita la struttura del sistema informativo al suo massimo grado di astrazione. Attraverso la trasposizione dei requisiti informativi in un modello concettuale, si delinea un'architettura logica svincolata dalle specifiche tecnologiche del DBMS. Tale fase si concretizza in un Class Diagram UML, finalizzato a rappresentare le entità cardine del dominio applicativo, le reciproche interazioni e i vincoli di integrità che regolano il sistema.

2.1 Class Diagram



2.3 Ristrutturazione del Class Diagram

Prima di procedere alla traduzione del Class Diagram nel modello relazionale, è stata effettuata un'analisi di ristrutturazione per ottimizzare lo schema e rimuovere eventuali ambiguità che potrebbero compromettere l'efficienza della base di dati.

2.3.1 Analisi delle ridondanze

Dall'analisi delle entità e delle associazioni non sono emerse ridondanze informative. Tutti i dati persistiti (come titoli, descrizioni e scadenze) sono indipendenti e non derivabili da altri attributi. Si è scelto deliberatamente di non inserire attributi derivati (es. "numero task totali" in una bacheca) per delegare eventuali conteggi a funzioni di aggregazione SQL in fase di interrogazione.

2.3.2 Eliminazione di attributi composti e multivalore

Si è verificato che tutti gli attributi definiti nel dizionario delle classi siano **atomici**.

- Non sono presenti attributi composti (es. la data è gestito come tipo di dato nativo del DBMS).
- Non sono presenti attributi multivalore (ogni campo contiene un singolo valore informativo). Pertanto, non è stata necessaria la creazione di tabelle ausiliarie per la scomposizione di attributi complessi.

2.3.3 Trattamento delle gerarchie

Il modello non presenta gerarchie o generalizzazioni. Le entità sono state modellate come classi distinte e autonome, rendendo lo schema lineare e direttamente traducibile in relazioni fisiche.

2.4 Dizionario delle classi

Classe	***Descrizione***	***Attributi***
users	Rappresenta gli utenti registrati che accedono al sistema per gestire le proprie bacheche.	**username** (String): identificativo univoco dell'utente. **password** (String): credenziali di accesso associate all'account. **created_at** (Timestamp): Registra automaticamente la data e l'ora di creazione del record.

		updated_at (Timestamp): Registra automaticamente la data e l'ora dell'ultimo aggiornamento del record.
boards	Modella le bacheche in cui l'utente organizza i propri todo	**title** (String): titolo della bacheca. **description** (String): testo che descrive le finalità della bacheca. **created_at** (Timestamp): Registra automaticamente la data e l'ora di creazione del record. **updated_at** (Timestamp): Registra automaticamente la data e l'ora dell'ultimo aggiornamento del record.
todos	Rappresenta la singola attività creata da un utente	**title** (String): nome sintetico dell'attività. **description** (String): testo dettagliato riguardante il task. **expiry_date** (Timestamp): data e ora entro cui completare l'attività. **url_link** (String): collegamento ipertestuale correlato. **image_path** (String): percorso locale dell'immagine associata. **background_color** (String): codice colore, in esadecimale, per la visualizzazione grafica.

			is_completed (Boolean): stato del task (completato/non completato). **position_order** (Integer): valore numerico che definisce l'ordine visivo nella bacheca. **created_at** (Timestamp): Registra automaticamente la data e l'ora di creazione del record. **updated_at** (Timestamp): Registra automaticamente la data e l'ora dell'ultimo aggiornamento del record.

2.5 Dizionario delle associazioni

Nome Associazione	***Descrizione***	***Classi Coinvolte***	***Attributo***
Gestione	Esprime il legame di proprietà tra l'utente e le proprie bacheche.	**User** [0..*] (ruolo: gestisce): indica che un utente può amministrare diverse bacheche personali. **Board** [1] (ruolo: è gestito): indica che ogni bacheca deve essere riconducibile ad un unico proprietario.	

Composizione	Definisce il vincolo di appartenenza dei todo ad una specifica bacheca.	**Board** [1] (ruolo: contiene): rappresenta il contenitore logico dei task. **Todo** [0..*] (ruolo: contenuti): indica l'insieme dei task ospitati in una bacheca. L'uso della composizione implica che la cancellazione della bacheca comporti la rimozione di tutti i Todo associati.	
Condivisione	Gestisce la natura collaborativa dei task tramite una relazione molti-a-molti.	**User** [1..] (ruolo: condiviso): indica l'insieme dei collaboratori che hanno accesso al task. **Todo** [0..] (ruolo: accesso): indica il task reso visibile ai collaboratori.	position_order (Integer): permette ad ogni collaboratore di definire un ordine di visualizzazione indipendente. **assigned_at** (Timestamp): Indica il momento in cui è avvenuta la condivisione del task tra proprietario e collaboratore.

2.6 Dizionario delle enumerazioni

Enumerazione	***Descrizione***	***Valori***
priority_level	Definisce i livelli di priorità che possono essere assegnati a un todo per organizzarne la rilevanza visiva.	**ALTA**: Indica un'attività prioritaria che richiede attenzione immediata. **MEDIA**: Rappresenta un task con urgenza standard. **BASSA**: Indica attività secondarie o non urgenti.

2.7 Dizionario dei vincoli

Vincolo	***Tipo***	***Descrizione***
Unicità Username	Intrarelazionale	Non possono esistere due utenti diversi con lo stesso username.
Titolo bacheca obbligatorio	Intrarelazionale	Ogni bacheca gestita da un utente deve avere un titolo non nullo.
Integrità Posizionamento	Intrarelazionale	Il valore dell'attributo position_order deve essere un intero positivo non nullo per garantire l'ordinamento sequenziale dei todo.
Eliminazione a Cascata dei todo	Interrelazionale	La cancellazione di una bacheca comporta l'eliminazione automatica di tutti i todo in essa contenuti (Vincolo di Composizione).
Eliminazione a Cascata dei todo condivisi	Intrarelazionale	La cancellazione di una bacheca comporta l'eliminazione automatica di tutti i todo condivisi.
Consistenza Scadenza	N-upla	Se presente, la data di scadenza di un todo deve

		essere uguale o successiva alla data di creazione del task stesso.
Unicità Condivisione	Interrelazionale	Lo stesso todo non può essere condiviso più volte con il medesimo utente collaboratore.
Limitazione Proprietario	Interrelazionale	L'autore del ToDo è l'unico utente autorizzato ad aggiungere o rimuovere altri collaboratori dalla lista di condivisione.
Stato Completamento	Dominio	L'attributo is_completed può assumere solo i valori booleani Vero o Falso (di default Falso).
Formato Username	Dominio	Definisce i requisiti strutturali per l'username
Integrità Hash Password	Dominio	Garantisce che la password memorizzata sia stata correttamente processata tramite algoritmo di hashing (es. BCrypt), verificando che la stringa risultante abbia una lunghezza fissa di 60 caratteri.
Obbligatorietà Credenziali	Intrarelazionale	Impone che i campi username e password non assumano mai valori nulli, garantendo l'integrità del processo di autenticazione.
Inibizione Renaming Bacheche Condivise	Interrelazionale	Impedisce la modifica del titolo delle bacheche qualora esse contengano todo condivisi. Questo garantisce che la bacheca di destinazione per i collaboratori rimanga coerente con la logica di sistema.
Allineamento Automatico Bacheche	Interrelazionale	Qualora l'utente proprietario sposti un ToDo condiviso in

		una bacheca con un determinato titolo (es. "Lavoro"), un trigger verifica se tale bacheca esista già nel profilo di ogni collaboratore. In caso negativo, il sistema provvede alla creazione automatica della bacheca mancante per garantire che il task rimanga sempre categorizzato correttamente per tutti gli utenti coinvolti.
Partecipazione Totale Bacheca	Interrelazionale	Ogni bacheca deve essere necessariamente associata ad un unico e solo utente esistente, garantendo che non esistano bacheche senza proprietario.
Partecipazione Totale Todo	Interrelazionale	Ogni todo deve essere necessariamente legato ad una bacheca esistente. Un todo non può esistere nel sistema se non è collocato all'interno di una bacheca fisica.
Inizzializzazione Account	Business Rule	Alla registrazione di un nuovo utente, il sistema dovrà provvedere alla creazione automatica di tre bacheche standard (Università, Lavoro, Tempo Libero) associate al nuovo utente registrato.
Eliminazione a cascata Utente	Interrelazionale	La cancellazione di un utente comporta alla rimozione automatica di tutte le bacheche ad esso associate, ed anche di tutti i todo che gli sono stati condivisi.

3.Progettazione Logica

In questo capitolo viene illustrato il passaggio dal modello concettuale al modello logico-relazionale. L'obiettivo della progettazione logica è tradurre le entità, le relazioni e i vincoli emersi nelle fasi precedenti in una struttura di tabelle ottimizzata per il DBMS PostgreSQL. Durante questa fase, le associazioni vengono trasformate in vincoli di chiave esterna (Foreign Key) e le entità in relazioni, garantendo la minima ridondanza dei dati e il rispetto delle forme normali.

Per la definizione delle chiavi primarie, si è scelto di adottare delle chiavi artificiali (o surrogate) di tipo intero per ogni tabella. Questa scelta permette di svincolare l'identificazione univoca dei record da attributi che potrebbero subire variazioni nel tempo (come lo username) e di ottimizzare le prestazioni del DBMS nelle operazioni di join e di indicizzazione, riducendo lo spazio di memorizzazione rispetto all'uso di chiavi naturali basate su stringhe.

Per garantire una corretta tracciabilità delle informazioni, è stata prevista per quasi tutte le tabelle l'introduzione degli attributi di audit created_at e updated_at. Tali campi permettono di storicizzare automaticamente il momento dell'inserimento e dell'ultima modifica di ogni record. Un'eccezione è rappresentata dalla tabella di relazione shared_todos, dove è presente unicamente l'attributo assigned_at, necessario per registrare il timestamp esatto in cui un'attività è stata condivisa con un determinato collaboratore.

3.1 Mapping delle associazioni

La traduzione dal Class Diagram allo schema relazionale ha seguito le seguenti regole di derivazione:

- **Relazioni 1:N (Gestione e Composizione):** Le associazioni tra User e Board, e tra Board e Todo, sono state modellate inserendo la chiave primaria del lato "1" come chiave esterna (FK) nella tabella del lato "N". Questo garantisce che ogni bacheca appartenga a un utente e ogni task a una specifica bacheca.
- **Relazione M:N (Condivisione):** L'associazione molti-a-molti tra User e Todo è stata risolta mediante l'introduzione di una tabella di relazione denominata shared_todos. Questa tabella non solo permette il legame tra più utenti e lo stesso task, ma ospita l'attributo position_order per consentire un ordinamento personalizzato per ogni collaboratore.

3.2 Schema logico

LEGENDA:

- Le chiavi primarie sono in grassetto sottolineato.
- Le chiavi esterne sono in corsivo.

users (**user_id**, username, password, created_at, updated_at)

boards (**board_id**, title, description, created_at, updated_at, *user_id*)
user_id → **users**.user_id

todos (**todo_id**, title, description, expiry_date, url_link, image_path, background_color, position_order, priority, is_completed, created_at, updated_at, *board_id*)
board_id → **boards**.bord_id

shared_todos (todo_id, user_id, position_order, assigned_at)
todo_id → **todos**.todo_id
user_id → **users**.user_id

4.Progettazione Fisica

In questo capitolo viene riportata l'implementazione dello schema logico precedentemente descritto all'interno del DBMS **PostgreSQL**. Vengono definite le tabelle con i relativi vincoli intrarelazionali (Primary Keys, Check, Not Null) e interrelazionali (Foreign Keys), oltre alle strutture avanzate come trigger e funzioni per l'automazione della logica di business.

4.1 Definizione Tabelle

In questa sezione viene riportata l'implementazione fisica dello schema logico nel DBMS PostgreSQL. Per ogni tabella è stata prevista l'adozione di vincoli di integrità e di tipi di dato coerenti con i requisiti.

4.1.1 Definizione della Tabella users

La tabella memorizza le credenziali di accesso degli utenti. L'identificativo user_id è implementato come SERIAL PRIMARY KEY per delegare al DBMS la generazione automatica di una chiave surrogata univoca. Per l'attributo username è stato imposto il vincolo UNIQUE per evitare duplicati (**Vincolo unicità Username**) e NOT NULL per assicurarne l'obbligatorietà (**Vincolo obbligatorietà credenziali**), supportato da un vincolo CHECK che ne valida il formato e la lunghezza (**Vincolo formato Username**). La password è anch'essa obbligatoria (**Vincolo obbligatorietà credenziali**) ed è soggetta a un controllo sulla lunghezza esatta (60 caratteri) per garantire che nel database venga memorizzata esclusivamente la versione protetta tramite algoritmo di hashing (**Vincolo di Integrità Hash Password**). Infine, i timestamp created_at e updated_at sfruttano la clausola DEFAULT

CURRENT_TIMESTAMP per storicizzare in modo trasparente il momento della registrazione e dell'ultima modifica.

```
CREATE TABLE public.users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT check_hash_length CHECK (length(password) = 60),
    CONSTRAINT check_username_format CHECK (
        length(username) >= 3 AND
        length(username) <= 20 AND
        username ~ '^[a-zA-Z0-9_.-]+$'
    )
);
```

4.1.2 Definizione della Tabella boards

Questa tabella rappresenta i contenitori logici dei task. Il board_id funge da chiave primaria artificiale. L'attributo title è obbligatorio (NOT NULL) (**Vincolo Titolo bacheca obbligatorio**), mentre la description è opzionale. È impostata la chiave esterna user_id, che lega la bacheca all'utente proprietario (**Vincolo Partecipazione totale bacheca**), con la clausola ON DELETE CASCADE che grantisce, in caso di eliminazione di un account utente, tutte le bacheche a lui intestate vengano automaticamente rimosse dal sistema, preservando l'integrità referenziale (**Eliminazione a cascata Utente**). I campi timestamp tracciano il ciclo di vita del record, in analogia con la tabella utenti.

```
CREATE TABLE public.boards (
    board_id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    user_id INTEGER NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_board_user FOREIGN KEY (user_id)
        REFERENCES public.users(user_id) ON DELETE CASCADE
);
```

4.1.3 Definizione della Tabella todos

Costituisce l'entità centrale del sistema, modellando le singole attività. Per facilitare l'inserimento dei dati lato applicativo, sono stati definiti alcuni valori di default: il background_color assume il colore bianco (#FFFFFF), lo stato is_completed è inizializzato a FALSE (**Vincolo Stato Completamento**) e la priorità riceve il valore minimo (BASSA) basato

sul tipo enumerativo priority_level. L'attributo position_order NOT NULL è essenziale per garantire la stabilità dell'ordinamento nella User Interface (**Vincolo Integrità posizionamento**). La chiave esterna board_id impone la rigida dipendenza logica del task dalla propria bacheca (**Vincolo Partecipazione Totale Todo**), con la clausola ON DELETE CASCADE che garantisce la rimozione dei task alla cancellazione del contenitore (**Vincolo Eliminazione a Cascata dei todo).** Inoltre, il vincolo check_expiry_after_created previene l'inserimento di scadenze incoerenti o collocate nel passato (con una tolleranza di 1 minuto rispetto alla creazione) (**Vincolo Consistenza scadenza**).

```sql
-- Definizione del tipo enumerativo
CREATE TYPE public.priority_level AS ENUM ('ALTA', 'MEDIA', 'BASSA');

CREATE TABLE public.todos (
    todo_id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    description TEXT,
    expiry_date TIMESTAMP,
    url_link TEXT,
    image_path TEXT,
    background_color VARCHAR(7) DEFAULT '#FFFFFF',
    is_completed BOOLEAN DEFAULT FALSE,
    position_order INTEGER DEFAULT 1 NOT NULL,
    priority public.priority_level DEFAULT 'BASSA',
    board_id INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_todo_board FOREIGN KEY (board_id)
        REFERENCES public.boards(board_id) ON DELETE CASCADE,
    CONSTRAINT check_expiry_after_created
        CHECK (expiry_date >= (created_at - interval '1 minute') OR expiry_date IS NULL)
);
```

4.1.4 Definizione della Tabella shared_todos

Questa tabella ponte risolve l'associazione molti-a-molti tra utenti e task. L'identificatore univoco è costituito da una chiave primaria composta (todo_id, user_id), che funge implicitamente da vincolo di unicità: garantisce infatti che il medesimo task non possa essere condiviso più volte con lo stesso utente (**Vincolo unicità condivisione**). Entrambe le chiavi esterne presentano la clausola ON DELETE CASCADE: quella sul todo_id assicura la pulizia dei riferimenti alla rimozione del task originale (**Vincolo Eliminazione a Cascata dei todo condivisi**), mentre quella sul user_id gestisce la rimozione delle condivisioni alla cancellazione del profilo collaboratore (**Vincolo Eliminazione a Cascata Utente**). L'attributo assigned_at memorizza il timestamp esatto in cui avviene la condivisione, mentre l'attributo position_order (DEFAULT 1 NOT NULL) consente a ciascun partecipante di posizionare visivamente il task in modo indipendente dall'autore originale (**Vincolo Integrità Posizionamento**).

```sql
CREATE TABLE public.shared_todos (
    todo_id INTEGER NOT NULL,
    user_id INTEGER NOT NULL,
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    position_order INTEGER DEFAULT 1 NOT NULL,
    PRIMARY KEY (todo_id, user_id), -- Chiave composta
    CONSTRAINT fk_shared_todo FOREIGN KEY (todo_id)
        REFERENCES public.todos(todo_id) ON DELETE CASCADE,
    CONSTRAINT fk_shared_user FOREIGN KEY (user_id)
        REFERENCES public.users(user_id) ON DELETE CASCADE
);
```

4.2 Definizione Viste

Per semplificare le operazioni di interrogazione lato applicativo e centralizzare la logica di recupero dei dati, si è scelto di implementare una vista logica, permettendo di astrarre la complessità dei join multipli e di fornire all'interfaccia grafica un set di risultati unificato e pronto per la visualizzazione.

4.2.1 Definizione della vista v_user_visible_todos

La vista v_user_visible_todos ha lo scopo di estrarre la totalità dei task visibili da un determinato utente (identificato come viewer_id), indipendentemente dal fatto che ne sia il creatore originale o un semplice collaboratore.

Per ottenere questo risultato, la vista utilizza l'operatore UNION ALL per concatenare due insiemi di dati distinti, normalizzandone lo schema di uscita:

1. **Task di Proprietà (OWNER):** Il primo blocco della query seleziona i ToDo creati direttamente dall'utente. In questo caso, l'attributo position_order e il riferimento alla bacheca (board_id) vengono prelevati direttamente dalle tabelle principali (todos e boards). A questi record viene assegnato staticamente il ruolo testuale 'OWNER'.

2. **Task Condivisi (SHARED):** Il secondo blocco gestisce i task di cui l'utente è collaboratore. La query si interfaccia con la tabella associativa shared_todos per estrarre il position_order personalizzato dell'utente visualizzatore. La criticità relativa alla collocazione del task viene risolta tramite un join condizionale: il sistema individua dinamicamente la bacheca del collaboratore (b_shared) che possiede lo stesso titolo di quella in cui il proprietario originale (u_owner) ha inserito il ToDo. A questi record viene assegnato il ruolo 'SHARED'.

Questa astrazione garantisce che l'applicativo Java riceva una lista omogenea di ToDo, delegando interamente al DBMS la risoluzione delle gerarchie di condivisione e dei posizionamenti indipendenti.

```sql
CREATE VIEW public.v_user_visible_todos AS
 SELECT t.todo_id,
    t.title,
    t.description,
    t.expiry_date,
    t.is_completed,
    t.priority,
    t.board_id,
    t.position_order,
    t.background_color,
    t.image_path,
    t.url_link,
    b.title AS board_title,
    u.user_id AS viewer_id,
    u.username AS owner_username,
    'OWNER'::text AS role
   FROM ((public.todos t
     JOIN public.boards b ON ((t.board_id = b.board_id)))
     JOIN public.users u ON ((b.user_id = u.user_id)))
UNION ALL
 SELECT t.todo_id,
    t.title,
    t.description,
    t.expiry_date,
    t.is_completed,
    t.priority,
    b_shared.board_id,
    st.position_order,
    t.background_color,
    t.image_path,
    t.url_link,
    b_shared.title AS board_title,
    st.user_id AS viewer_id,
    u_owner.username AS owner_username,
    'SHARED'::text AS role
   FROM ((((public.todos t
     JOIN public.shared_todos st ON ((t.todo_id = st.todo_id)))
     JOIN public.boards b_owner ON ((t.board_id = b_owner.board_id)))
     JOIN public.users u_owner ON ((b_owner.user_id = u_owner.user_id)))
     JOIN public.boards b_shared ON (((b_shared.user_id = st.user_id) AND
((b_shared.title)::text = (b_owner.title)::text))));
```

4.3 Definizione Trigger

In questa sezione vengono documentati i Trigger che automatizzano la gestione dei dati e garantiscono il rispetto dei vincoli definiti in fase di analisi.

4.3.1 Inizializzazione automatica delle bacheche (Vincolo Inizializzazione Account)

Come previsto dai requisiti funzionali, ogni nuovo utente deve disporre di tre bacheche predefinite al momento della registrazione. Per automatizzare tale è stato implementato un trigger di tipo AFTER INSERT sulla tabella users. La funzione associata esegue tre operazioni di inserimento nella tabella boards, collegandole al nuovo user_id generato (identificato dalla variabile NEW).

```
-- Definisce la funzione per la creazione delle bacheche di default
CREATE FUNCTION public.create_default_boards() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
    -- Inserisce la bacheca dedicata agli esami
    INSERT INTO boards (title, description, user_id)
    VALUES ('Universit' || U&'\00E0', 'Gestione dei corsi, esami da sostenere e scadenze
dei progetti universitari.', NEW.user_id);

    -- Inserisce la bacheca dedicata alle attivita professionali
    INSERT INTO boards (title, description, user_id)
    VALUES ('Lavoro', 'Task professionali, scadenze ufficio e obiettivi di carriera.',
NEW.user_id);

    -- Inserisce la bacheca dedicata agli hobby e alla vita privata
    INSERT INTO boards (title, description, user_id)
    VALUES ('Tempo liber' || U&'\00E0', 'Hobby, sport, viaggi e attivit' || U&'\00E0' || '
personali da svolgere nel tempo libero.', NEW.user_id);

    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella users dopo l'inserimento di un record
CREATE TRIGGER trigger_add_default_boards
AFTER INSERT ON public.users
FOR EACH ROW
EXECUTE FUNCTION public.create_default_boards();
```

4.3.2 Gestione automatica del posizionamento (Vincolo Integrità Posizionamento)

Per garantire la coerenza dell'ordinamento visivo dei task all'interno delle bacheche, è stato implementato un trigger di tipo BEFORE INSERT sulla tabella todos. Tale automazione evita conflitti di posizionamento durante la creazione di nuovi task, assicurando che ogni elemento venga aggiunto in coda alla sequenza esistente.

La funzione check_todo_position_logic opera seguendo questi passaggi:

1. **Identificazione del proprietario:** Recupera l'identificativo dell'utente proprietario della bacheca di destinazione.
2. **Calcolo della posizione massima:** Interroga la vista unificata v_user_visible_todos, così da considerare anche i todo condivisi, per determinare il valore massimo dell'attributo position_order attualmente presente in quella specifica bacheca per quel proprietario.
3. **Assegnazione incrementale:** Imposta il valore di position_order del nuovo record (NEW) incrementando di un'unità il massimo precedentemente calcolato (utilizzando COALESCE per gestire il caso di bacheca vuota).

```
-- Definisce la funzione per il calcolo automatico della posizione del task
CREATE FUNCTION public.check_todo_position_logic() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE
    v_max_pos INTEGER;
    v_owner_id INTEGER;
BEGIN
    -- Recupera l'identificativo del proprietario della bacheca di destinazione
    SELECT user_id INTO v_owner_id FROM public.boards WHERE board_id = NEW.board_id;

    -- Determina la posizione massima attuale interrogando la vista
    SELECT COALESCE(MAX(position_order), 0) INTO v_max_pos
    FROM public.v_user_visible_todos
    WHERE board_id = NEW.board_id AND viewer_id = v_owner_id;

    -- Assegna al nuovo task la posizione successiva all'ultima disponibile
    NEW.position_order := v_max_pos + 1;

    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella todos prima di ogni operazione di inserimento
CREATE TRIGGER trg_fix_todo_position_insert
BEFORE INSERT ON public.todos
FOR EACH ROW
EXECUTE FUNCTION public.check_todo_position_logic();
```

4.3.3 Gestione dell'ordine nelle condivisioni (Vincolo Integrità Posizionamento)

Per garantire che i task condivisi siano immediatamente visibili nella bacheca del destinatario, è stata implementata una logica di posizionamento prioritario tramite un trigger di tipo BEFORE INSERT sulla tabella shared_todos. A differenza dei task proprietari, i task condivisi vengono inseriti forzatamente nella prima posizione visiva.

La funzione check_shared_todo_position_logic esegue le seguenti operazioni:

1. **Risoluzione del contesto:** Individua il titolo della bacheca originale del task e cerca la bacheca corrispondente (con lo stesso titolo) nel profilo del destinatario della condivisione.
2. **Scorrimento degli elementi esistenti:** Qualora la bacheca di destinazione esista, il sistema esegue un'operazione di traslazione (UPDATE) incrementando di un'unità il position_order di tutti i task (sia proprietari che già condivisi) presenti in quel contenitore per quell'utente. Questo "libera" la posizione numero 1.
3. **Assegnazione della posizione:** Imposta il valore di position_order del nuovo record di condivisione (NEW) al valore fisso 1.

Questa automazione preserva l'integrità dell'ordinamento sequenziale, evitando sovrapposizioni.

```
-- Definisce la funzione per il posizionamento prioritario dei todo condivisi
CREATE FUNCTION public.check_shared_todo_position_logic() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE
    v_target_board_title VARCHAR(100);
    v_target_board_id INTEGER;
BEGIN
    -- Recupera il titolo della bacheca di origine del todo
    SELECT b.title INTO v_target_board_title
    FROM public.todos t
    JOIN public.boards b ON t.board_id = b.board_id
    WHERE t.todo_id = NEW.todo_id;

    -- Individua l'ID della bacheca omologa per l'utente ricevente
    SELECT board_id INTO v_target_board_id
    FROM public.boards
    WHERE title = v_target_board_title AND user_id = NEW.user_id;

    -- Verifica l'esistenza della bacheca prima di procedere allo scorrimento
    IF v_target_board_id IS NOT NULL THEN
        -- Incrementa la posizione dei todo di cui l'utente e titolare
        UPDATE public.todos
        SET position_order = position_order + 1
```

```
        WHERE board_id = v_target_board_id;

        -- Incrementa la posizione degli altri todo gia condivisi nella medesima bacheca
        UPDATE public.shared_todos st
        SET position_order = st.position_order + 1
        FROM public.todos t
        JOIN public.boards b ON t.board_id = b.board_id
        WHERE st.todo_id = t.todo_id
          AND st.user_id = NEW.user_id
          AND b.title = v_target_board_title;
    END IF;

    -- Assegna la prima posizione al todo appena condiviso
    NEW.position_order := 1;

    -- Restituisce il record pronto per la memorizzazione
    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella delle condivisioni prima dell'inserimento
CREATE TRIGGER trg_fix_shared_position_insert
BEFORE INSERT ON public.shared_todos
FOR EACH ROW
EXECUTE FUNCTION public.check_shared_todo_position_logic();
```

4.3.4 Allineamento automatico dei contenitori condivisi (Vincolo Allineamento Automatico Bacheche)

Al fine di garantire la coerenza dell'organizzazione dei dati tra diversi utenti, è stata implementata una logica di sincronizzazione automatica delle bacheche. Il meccanismo interviene tramite un trigger di tipo AFTER INSERT sulla tabella shared_todos ogni volta che un todo viene condiviso con un nuovo collaboratore.

La funzione check_and_create_board_on_share opera secondo una logica di verifica e creazione:

1. **Recupero del contesto originale:** Attraverso una join tra todos e boards, il sistema individua il titolo della bacheca in cui il proprietario ha archiviato il task.
2. **Verifica dell'esistenza:** Viene eseguita una query sulla tabella boards per controllare se il destinatario della condivisione possieda già un contenitore con il medesimo titolo.
3. **Creazione dinamica:** Qualora il controllo restituisca esito negativo, il sistema provvede automaticamente all'inserimento di una nuova bacheca per il collaboratore, utilizzando il titolo originale e il user_id del destinatario.

Questa automazione è fondamentale per il corretto funzionamento della vista v_user_visible_todos, la quale si basa sulla corrispondenza dei titoli tra le bacheche del proprietario e quelle del collaboratore per mostrare i contenuti.

```
-- Definisce la funzione per garantire l'esistenza della bacheca di destinazione
CREATE FUNCTION public.check_and_create_board_on_share() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE
    origin_board_title VARCHAR(100);
    target_has_board BOOLEAN;
BEGIN
    -- Individua il titolo della bacheca originale del task oggetto di condivisione
    SELECT b.title INTO origin_board_title
    FROM todos t JOIN boards b ON t.board_id = b.board_id
    WHERE t.todo_id = NEW.todo_id;

    -- Verifica se il destinatario possiede gia un contenitore con lo stesso titolo
    SELECT EXISTS (
        SELECT 1 FROM boards
        WHERE user_id = NEW.user_id AND title = origin_board_title
    ) INTO target_has_board;

    -- Provvede alla creazione della bacheca qualora non risulti esistente
    IF NOT target_has_board THEN
        INSERT INTO boards (title, user_id) VALUES (origin_board_title, NEW.user_id);
    END IF;

    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella shared_todos per intervenire dopo ogni condivisione
CREATE TRIGGER trigger_ensure_board_exists
AFTER INSERT ON public.shared_todos
FOR EACH ROW
EXECUTE FUNCTION public.check_and_create_board_on_share();
```

4.3.5 Sincronizzazione dinamica allo spostamento (Vincolo Allineamento Automatico Bacheche)

Per garantire che un task condiviso resti visibile a tutti i collaboratori anche a seguito di una riorganizzazione da parte del proprietario, è stato implementato un trigger di tipo AFTER UPDATE sulla colonna board_id della tabella todos. Tale automazione interviene ogni volta che un ToDo viene spostato in una bacheca differente da quella di origine.

La funzione check_and_create_board_on_move esegue una scansione ricorsiva della rete di condivisione:

1. **Verifica della modifica:** Il trigger si attiva solo se l'identificativo della bacheca (board_id) è effettivamente cambiato.
2. **Iterazione sui collaboratori:** Tramite un ciclo FOR, il sistema individua tutti gli utenti presenti nella tabella shared_todos legati a quel determinato todo.
3. **Controllo e creazione:** Per ogni collaboratore, si verifica l'esistenza di una bacheca con lo stesso titolo della nuova destinazione. In caso di assenza, la bacheca viene creata istantaneamente, assicurando che la vista v_user_visible_todos possa continuare a mappare correttamente il todo nel profilo del collaboratore.

```
-- Definisce la funzione per la sincronizzazione delle bacheche durante lo spostamento
CREATE FUNCTION public.check_and_create_board_on_move() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE
    new_board_title VARCHAR(100);
    collaborator_record RECORD;
BEGIN
    -- Interrompe l'esecuzione se l'identificativo della bacheca non e variato
    IF NEW.board_id = OLD.board_id THEN RETURN NEW; END IF;

    -- Recupera il titolo della nuova bacheca di destinazione
    SELECT title INTO new_board_title FROM boards WHERE board_id = NEW.board_id;

    -- Itera su tutti i collaboratori associati al todo spostato
    FOR collaborator_record IN
        SELECT user_id FROM shared_todos WHERE todo_id = NEW.todo_id
    LOOP
        -- Crea la bacheca omologa per il collaboratore qualora non esistente
        IF NOT EXISTS (
            SELECT 1 FROM boards
            WHERE user_id = collaborator_record.user_id AND title = new_board_title
        ) THEN
            INSERT INTO boards (title, user_id) VALUES (new_board_title,
collaborator_record.user_id);
        END IF;
    END LOOP;

    RETURN NEW;
END;
$$;

-- Attiva la funzione esclusivamente all'aggiornamento del campo board_id
```

```
CREATE TRIGGER trigger_ensure_board_on_move
AFTER UPDATE OF board_id ON public.todos
FOR EACH ROW
EXECUTE FUNCTION public.check_and_create_board_on_move();
```

4.3.6 Gestione delle eliminazioni delle condivisioni (Vincolo Eliminazione a Cascata dei todo condivisi)

Per garantire la corretta rimozione dei legami di collaborazione a seguito dell'eliminazione di una bacheca, è stato implementato un trigger di tipo BEFORE DELETE sulla tabella boards. Questo trigger garantisce che, prima della rimozione fisica di una bacheca, l'utente "esca" da tutte le condivisioni associate a quella particolare bacheca.

La funzione leave_shares_on_board_delete agisce sulla tabella di legame:

1. **Identificazione del contesto:** La funzione recupera il titolo della bacheca che sta per essere rimossa.
2. **Rimozione selettiva:** Viene eseguita un'operazione di DELETE sulla tabella shared_todos per tutti i record in cui l'utente corrente è coinvolto, limitatamente ai todo che appartengono logicamente a bacheche con lo stesso titolo.
3. **Integrità referenziale:** Questa operazione precede la cancellazione della bacheca stessa, evitando la permanenza di record orfani e garantendo che la rimozione di un contenitore personale comporti anche la rinuncia alla visualizzazione dei task condivisi in quella specifica categoria.

```
-- Definisce la funzione per la rimozione delle condivisioni associate a una bacheca
CREATE FUNCTION public.leave_shares_on_board_delete() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
    -- Rimuove la partecipazione ai task condivisi dell'utente
    -- per tutte le attivita associate a bacheche con lo stesso titolo
    -- di quella in fase di eliminazione.
    DELETE FROM shared_todos
    WHERE user_id = OLD.user_id
    AND todo_id IN (
        SELECT t.todo_id
        FROM todos t
        JOIN boards b ON t.board_id = b.board_id
        WHERE b.title = OLD.title
    );
```

```
    -- Restituisce il record bacheca per procedere alla rimozione fisica
    RETURN OLD;
END;
$$;

-- Associa la funzione alla tabella boards prima dell'operazione di cancellazione
CREATE TRIGGER trigger_leave_shares
BEFORE DELETE ON public.boards
FOR EACH ROW
EXECUTE FUNCTION public.leave_shares_on_board_delete();
```

4.3.7 Inibizione della modifica del titolo (Vincolo Inibizione Renaming Bacheche Condivise)

Per garantire la stabilità dei collegamenti logici tra i diversi utenti, è stata implementata una restrizione sulla modifica dei titoli delle bacheche tramite un trigger di tipo BEFORE UPDATE sulla tabella boards. Tale vincolo assicura che il sistema di mappatura basato sulla corrispondenza dei nomi rimanga integro per tutta la durata della condivisione.

La funzione prevent_collaborator_rename applica la seguente logica di controllo:

1. **Verifica della variazione:** Il trigger interviene esclusivamente se il nuovo titolo (NEW.title) differisce da quello precedente (OLD.title).
2. **Identificazione dei task dipendenti:** Viene eseguita una scansione sulla tabella shared_todos per verificare se l'utente che sta tentando la modifica possieda, all'interno di quella bacheca, dei task che gli sono stati condivisi da terzi.
3. **Blocco dell'operazione:** Qualora venga rilevata la presenza di almeno un todo condiviso, il sistema solleva un'eccezione (RAISE EXCEPTION), impedendo l'aggiornamento. Questo obbliga il mantenimento del titolo originale, garantendo che il legame creato dal proprietario resti valido e visibile nella vista unificata.

```
-- Definisce la funzione per impedire la ridenominazione di bacheche con task condivisi
CREATE FUNCTION public.prevent_collaborator_rename() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE
    shared_task_count INTEGER;
BEGIN
    -- Consente l'aggiornamento se il titolo non ha subito variazioni
    IF NEW.title = OLD.title THEN
        RETURN NEW;
```

```
    END IF;

    -- Conta i task condivisi con l'utente corrente che dipendono dal titolo attuale
    SELECT COUNT(*) INTO shared_task_count
    FROM shared_todos st
    JOIN todos t ON st.todo_id = t.todo_id
    JOIN boards owner_board ON t.board_id = owner_board.board_id
    WHERE st.user_id = NEW.user_id
    AND owner_board.title = OLD.title;

    -- Impedisce la modifica se la bacheca funge da contenitore per task di altri utenti
    IF shared_task_count > 0 THEN
        RAISE EXCEPTION 'Impossibile rinominare: la bacheca contiene task condivisi. Solo
il proprietario originale puo variare il titolo del progetto.';
    END IF;

    -- Restituisce il record per autorizzare l'aggiornamento (se non ci sono vincoli)
    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella boards prima della modifica del titolo
CREATE TRIGGER trigger_prevent_rename
BEFORE UPDATE OF title ON public.boards
FOR EACH ROW
EXECUTE FUNCTION public.prevent_collaborator_rename();
```

4.3.8 Propagazione della ridenominazione (Vincolo Allineamento Automatico Bacheche)

Per garantire che la ridenominazione di una bacheca da parte del proprietario non interrompa la visibilità dei task per i collaboratori, è stata implementata una logica di creazione dinamica dei nuovi contenitori. Invece di modificare forzatamente le bacheche esistenti dei collaboratori, il sistema assicura che essi dispongano sempre di una bacheca con il nuovo titolo, preservando l'integrità della mappatura logica.

La funzione sync_board_title_on_rename opera nel seguente modo:

1. **Verifica del cambiamento:** Il trigger si attiva solo in caso di effettiva variazione del titolo.
2. **Identificazione dei collaboratori:** Viene estratto l'elenco univoco di tutti gli utenti che partecipano ai task contenuti nella bacheca rinominata.
3. **Assicurazione del contenitore:** Per ogni collaboratore, il sistema verifica se esiste già una bacheca con il nuovo titolo. In caso negativo, provvede a un'operazione di INSERT,

creando il nuovo contesto necessario affinché i task spostati restino visibili nella vista unificata.

```
-- Definisce la funzione per la creazione delle nuove bacheche a seguito di ridenominazione
CREATE OR REPLACE FUNCTION public.sync_board_title_on_rename() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
DECLARE
    collaborator_id INT;
BEGIN
    -- Interrompe l'esecuzione se il titolo della bacheca non e variato
    IF NEW.title = OLD.title THEN
        RETURN NEW;
    END IF;

    -- Itera su tutti i collaboratori associati ai task della bacheca rinominata
    FOR collaborator_id IN
        SELECT DISTINCT st.user_id
        FROM todos t
        JOIN shared_todos st ON t.todo_id = st.todo_id
        WHERE t.board_id = NEW.board_id
    LOOP
        -- Inserisce la nuova bacheca per il collaboratore solo se non gia presente
        INSERT INTO boards (title, user_id)
        SELECT NEW.title, collaborator_id
        WHERE NOT EXISTS (
            SELECT 1 FROM boards
            WHERE user_id = collaborator_id AND title = NEW.title
        );
    END LOOP;

    -- Restituisce il record bacheca aggiornato
    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella boards dopo l'aggiornamento del titolo
CREATE OR REPLACE TRIGGER trigger_sync_board_rename
AFTER UPDATE OF title ON public.boards
FOR EACH ROW
EXECUTE FUNCTION public.sync_board_title_on_rename();
```

4.3.9 Aggiornamento automatico dei timestamp di audit

Per garantire la precisione della storicizzazione dei dati, è stata implementata una logica di aggiornamento automatico per l'attributo updated_at presente nelle tabelle principali del

sistema. Questo meccanismo assicura che ogni operazione di modifica (UPDATE) venga tracciata temporalmente in modo trasparente e centralizzato.

La funzione update_timestamp opera in modo atomico:

1. **Intercettazione dell'aggiornamento:** Il trigger interviene prima che la modifica venga salvata fisicamente sul disco (BEFORE UPDATE).
2. **Sovrascrittura del campo:** Il sistema imposta il valore della colonna updated_at del record corrente (NEW) al timestamp corrente del server (CURRENT_TIMESTAMP).

```
-- Definisce la funzione per l'aggiornamento del timestamp di modifica
CREATE FUNCTION public.update_timestamp() RETURNS trigger
    LANGUAGE plpgsql
    AS $$
BEGIN
    -- Aggiorna l'attributo con la data e l'ora correnti
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$;

-- Associa la funzione alla tabella boards
CREATE TRIGGER set_timestamp_boards
BEFORE UPDATE ON public.boards
FOR EACH ROW
EXECUTE FUNCTION public.update_timestamp();

-- Associa la funzione alla tabella todos
CREATE TRIGGER set_timestamp_todos
BEFORE UPDATE ON public.todos
FOR EACH ROW
EXECUTE FUNCTION public.update_timestamp();

-- Associa la funzione alla tabella users
CREATE TRIGGER set_timestamp_users
BEFORE UPDATE ON public.users
FOR EACH ROW
EXECUTE FUNCTION public.update_timestamp();
```

4.4 Definizione Funzioni

Sono state predisposte delle funzioni SQL per filtraggi da eseguire sui todo, relativamente ai titoli, descrizioni o date.

4.4.1 Ricerca testuale (fn_search_todos)

Questa funzione esegue una ricerca "case-insensitive" su titoli e descrizioni, usando la funzione format() per costruire dinamicamente il pattern di ricerca. In input sono specificati id utente e titolo bacheca su cui effettuare la ricerca.

```
-- Esegue la ricerca testuale tramite l'ausilio della funzione format
CREATE OR REPLACE FUNCTION public.fn_search_todos(p_user_id INT, p_board_title VARCHAR,
p_query TEXT)
RETURNS SETOF public.v_user_visible_todos AS $$
BEGIN
    -- Restituisce i record applicando un filtro case-insensitive sulla bacheca indicata
    RETURN QUERY
    SELECT * FROM v_user_visible_todos
    WHERE viewer_id = p_user_id
    AND board_title = p_board_title
    AND (title ILIKE format('%%%s%%', p_query)
         OR description ILIKE format('%%%s%%', p_query));
END;
$$ LANGUAGE plpgsql;
```

4.4.2 Filtro scadenze giornata odierna (fn_get_due_today)

La funzione estrae la lista dei ToDo la cui scadenza coincide con la data corrente. Il confronto viene effettuato tramite il casting dell'attributo expiry_date al tipo DATE, così da ignorare le componenti orarie e confrontare esclusivamente il giorno, il mese e l'anno rispetto alla costante CURRENT_DATE.

```
-- Recupera i todo in scadenza oggi per una specifica bacheca e utente
CREATE OR REPLACE FUNCTION public.fn_get_due_today(p_user_id INT, p_board_title VARCHAR)
RETURNS SETOF public.v_user_visible_todos AS $$
BEGIN
    -- Restituisce i record della vista filtrando per data corrente e titolo bacheca
    RETURN QUERY
    SELECT * FROM v_user_visible_todos
    WHERE viewer_id = p_user_id
    AND board_title = p_board_title
    AND expiry_date::DATE = CURRENT_DATE;
END;
$$ LANGUAGE plpgsql;
```

4.4.3 Ricerca per data limite (fn_get_due_within)

Questa funzione permette di visualizzare i todo che scadono entro un intervallo temporale definito, partendo dalla data odierna fino a un limite superiore fornito in input (p_target_date).

```sql
-- Recupera i todo che scadono entro una data specifica (limite superiore)
CREATE OR REPLACE FUNCTION public.fn_get_due_within(p_user_id INT, p_board_title VARCHAR,
p_target_date DATE)
RETURNS SETOF public.v_user_visible_todos AS $$
BEGIN
    -- Seleziona i todo con scadenza compresa tra oggi e la data fornita in input
    RETURN QUERY
    SELECT * FROM v_user_visible_todos
    WHERE viewer_id = p_user_id
    AND board_title = p_board_title
    AND expiry_date::DATE >= CURRENT_DATE
    AND expiry_date::DATE <= p_target_date;
END;
$$ LANGUAGE plpgsql;
```

4.5 Note sulla gestione dell'ordinamento dei todo in una bacheca

In fase di progettazione fisica, si è valutata l'opportunità di implementare vincoli di unicità rigorosi sull'attributo *position_order* all'interno delle relazioni *todos* e *boards*. Tuttavia, a seguito di un'analisi del comportamento dinamico dell'interfaccia — basata su operazioni di *Drag & Drop* che coinvolgono spostamenti rapidi e multipli — si è scelto di delegare la responsabilità della coerenza sequenziale interamente allo strato applicativo.

L'implementazione di trigger di riordino lato DBMS è stata evitata per garantire la stabilità del sistema ed eliminare il rischio di posizioni duplicate. L'utilizzo di trigger per gestire l'aggiornamento della posizione potrebbe, infatti, innescare aggiornamenti ricorsivi o fenomeni di deadlock durante i salvataggi inviati dall'interfaccia grafica. Delegando il ricalcolo degli indici *position_order* alla logica di business in memoria, l'applicativo invia al database un set di dati già validato ed ordinato. Questa strategia permette di ottimizzare il carico computazionale sul server evitando l'esecuzione di trigger ridondanti per ogni singolo record spostato.